EXPERIMENT 4 - SERIAL COM IMU RFID

PART 1 - Real-Time Motion Tracking Using MPU6050 Sensor

PART 2 - Integrated Smart Access System

MCTA 3203 - MECHATRONICS SYSTEM INTEGRATION

DR. ZULKIFLI BIN ZAINAL ABIDIN

DR. WAHJU SEDIONO

SECTION 1

GROUP 11

MATRIC NUMBER	**NAME**
2316963	MUHAMMAD AIMAN ALFARUQ BIN NOREDAN
2316441	MUHAMMAD HARIZ BIN HELMI SHAMSUL
2316957	WAN SYAZMIE AKMAL BIN WAN SAAD

PART 1 - Real-Time Motion Tracking Using MPU6050 Sensor

Introduction

1. Overview of the Experiment's Purpose and Objectives

The first part of the experiment focuses on real-time motion tracking using the MPU6050 sensor connected to an Arduino microcontroller. The main purpose is to capture and analyze the sensor's accelerometer and gyroscope data to detect and visualize motion patterns, particularly circular hand gestures, in Python. This task aims to strengthen understanding of sensor data acquisition, serial communication, and real-time data visualization using Python libraries such as *matplotlib*. Through this experiment, students learn to process continuous motion data and interpret sensor readings effectively.

2. Background Information and Relevant Theory or Concepts

The MPU6050 is a 6-axis motion tracking device that integrates a 3-axis accelerometer and a 3-axis gyroscope into one compact chip. It operates based on microelectromechanical systems (MEMS) technology, which measures acceleration, orientation, and angular velocity in three-dimensional space. By interfacing with the Arduino via the I²C protocol, the sensor transmits continuous data that can be processed in Python for motion pattern recognition.

Motion detection applications rely heavily on sensor fusion algorithms and real-time plotting techniques to interpret dynamic movements. By using serial communication between Arduino and Python, motion trajectories can be visualized instantly, allowing for gesture recognition such as circular or linear movements. This task lays the foundation for intelligent motion sensing systems used in robotics, automation, and wearable technologies.

3. Hypothesis or Expected Outcomes

It is hypothesized that the MPU6050 sensor will successfully detect and track hand movement trajectories when integrated with Arduino and Python. The experiment is expected to produce a dynamic real-time plot that corresponds accurately to the user's motion. Furthermore, circular motion gestures should form clear loop-like patterns on the graph, confirming correct sensor calibration and serial data handling. The anticipated outcome is a stable and responsive motion detection system that demonstrates the accuracy and versatility of the MPU6050 sensor in embedded applications.

Materials & Equipment

- Hardware Required
- Arduino board
- MPU6050 sensor
- Jumper wires/breadboard
- USB cable
- Computer with Arduino IDE and Python installed

Experimental Setup

Circuit Setup

- The experiment used the **MPU6050 sensor** connected to the **Arduino Uno** via the **I²C communication interface**.
- The connections were made as follows:
  - **VCC → 5V**
  - **GND → GND**
  - **SDA → A4**
  - **SCL → A5**
- The **Arduino Uno** was connected to the **computer** through a **USB cable**, which provided both **power supply** and **serial communication**.
- The Arduino read data from the MPU6050 (accelerometer and gyroscope values) and sent them through the serial port to **Python** for further processing.
- In Python, the received data were processed and plotted dynamically using the **matplotlib** library.
- The setup was designed to perform the following tasks:
  - **Capture real-time accelerometer X-Y data.**
  - **Plot the motion dynamically using matplotlib.**
  - **Detect circular motion gesture.**
- The schematic shows the MPU6050 connected to the Arduino Uno with four jumper wires (VCC, GND, SDA, SCL), while the computer receives and visualizes the live motion data.

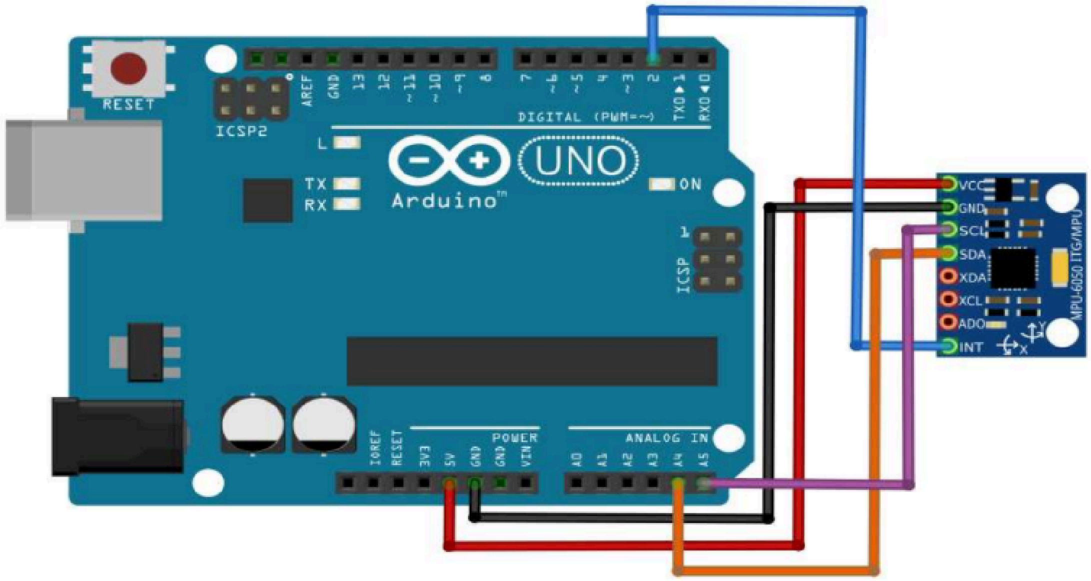


Figure 1: Circuit connection between MPU6050 and Arduino Uno for motion detection.

Experimental Setup

- The Arduino Uno was powered through the USB connection to the computer. The MPU6050 sensor was held or moved by hand to simulate motion in different directions.
- During operation:
  - The Arduino continuously read accelerometer and gyroscope data.
  - The Python program received the serial data stream and plotted the X-Y trajectory in real time.
- Circular hand movements were performed to verify the system's ability to detect specific motion patterns.
- The experimental setup helped observe the accuracy and responsiveness of the sensor as well as the smoothness of serial communication between Arduino and Python.

PART 2 - Integrated Smart Access System

Introduction

- **Overview of the Experiment's Purpose and Objectives**

The second part of the experiment aims to develop an integrated smart access control system that combines RFID authentication and motion-based verification. The system uses an RFID reader (MFRC522) to identify authorized users and verifies access through motion detection from the MPU6050 sensor before activating a servo motor and LED indicators. The main objective is to demonstrate mechatronic system integration involving sensors, actuators, and microcontrollers working together through serial communication between Arduino and Python. This task simulates real-world security systems that use multi-factor verification.

- **Background Information and Relevant Theory or Concepts**

The MFRC522 RFID module operates at 13.56 MHz and uses electromagnetic induction to read or write data to RFID tags without physical contact. Each RFID tag contains a unique identification code (UID), allowing selective access control when matched with pre-stored authorized IDs (Singh et al., 2020). This module communicates with Arduino using the SPI (Serial Peripheral Interface) protocol, enabling fast and reliable data transmission.

In this experiment, the RFID module is integrated with a servo motor and LED indicators to create a feedback-driven control system. When an authorized RFID card is detected and the correct motion gesture (verified via Python) is performed, the servo motor unlocks and the green LED lights up. If either condition fails, access is denied, and the red LED is activated. This system demonstrates embedded control logic, signal integration, and automation principles commonly applied in industrial and security applications (Random Nerd Tutorials, n.d.).

3. Hypothesis or Expected Outcomes

It is hypothesized that the integrated RFID and motion-activated access control system will provide accurate identification and secure access verification. The servo motor is expected to respond only when both the RFID tag is authorized and the detected motion gesture matches the predefined pattern. The LEDs will act as visual indicators of the access status. The expected outcome is a functional smart access system demonstrating how mechatronic integration enhances reliability, security, and automation efficiency.

Materials & equipments

- Arduino board
- RFID reader (MFRC522) + RFID tags/cards
- Servo motor
- Red & Green LEDs + resistors
- MPU6050 sensor (from Part 1)
- Jumper wires and breadboard

Experimental Setup

Circuit Setup

- The second task combined the RFID module (MFRC522), MPU6050 sensor, servo motor, and LED indicators with the Arduino Uno.
- The connection configuration was as follows:
    - RFID SS → D10
    - RFID RST → D8
    - Servo Signal → D9
    - Green LED → D4
    - Red LED → D3
    - MPU6050 → VCC (5V), GND, SDA (A4), SCL (A5)
- All components shared the same 5V power and GND from the Arduino.
- The RFID module communicated through SPI protocol, while the MPU6050 continued using the I²C interface.
- The computer communicated with the Arduino to send and receive serial commands during system operation.
- The schematic shows all modules connected to the Arduino, forming an integrated smart access control system.

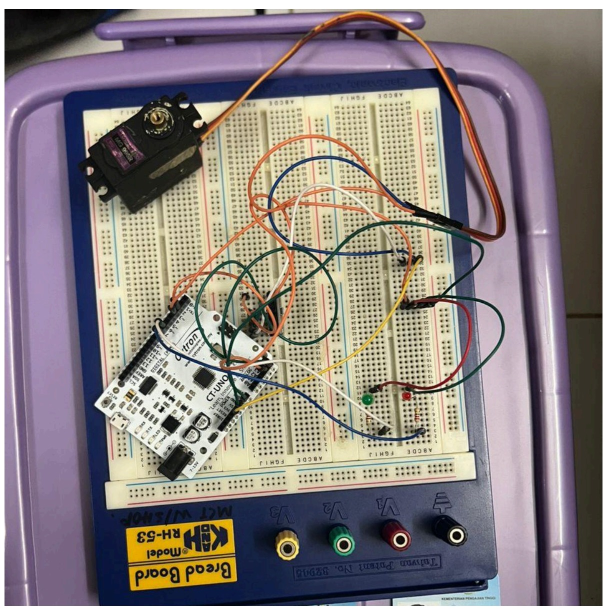


Figure 2: Circuit setup for RFID, MPU6050, servo motor, and LEDs with Arduino Uno.

Experimental Setup

- The Arduino Uno was connected to the computer running the Python control program.
- During testing:
    1. The RFID reader scanned the user's tag to verify authorization.
    2. If the RFID was valid, the system prompted the user to perform a motion gesture using the MPU6050 sensor.
    3. When the correct motion was detected (e.g., circular movement), a command was sent to unlock the servo motor and turn on the green LED.
    4. If the RFID was unauthorized or the motion was invalid, the red LED lit up, and the servo remained locked.

- This experiment demonstrated the integration of sensors, actuators, and communication protocols, showcasing how multi-factor verification can enhance security in mechatronic systems.
- The combined use of RFID authentication and motion detection reflects practical applications in smart access and automation systems.

Methodology

Part 1 – Real-Time Motion Tracking Using MPU6050 Sensor

Hardware Setup

- **Arduino Uno** is connected to the computer via USB.
- **LED** (optional) is connected to a digital pin for testing serial commands.
- No additional sensors are required for this part.

Software

Arduino :

1. Initialize the serial port using Serial.begin(9600).
2. Continuously send simple data such as text or sensor readings through the serial monitor.
3. Detect any incoming commands from Python using Serial.available().
4. Control an LED (or print feedback) when receiving a specific character (e.g., '1' to turn ON, '0' to turn OFF).

Python :

1. Use the pyserial library to open the same COM port at baud rate 9600.
2. Continuously read incoming data from Arduino and display it on the terminal.
3. Send control commands ('1' or '0') to Arduino through the serial connection.
4. Log the received data into a CSV file for confirmation.

Experimental Procedure

1. Upload the Arduino code that sends and receives serial data.

2. Run the Python script and verify that both can communicate through the COM port.
3. Observe data transmission from Arduino to Python.
4. Send control commands from Python and confirm Arduino response (e.g., LED turning ON/OFF).
5. Repeat several times to confirm reliable communication and correct baud rate.

Part 2 – Integrated Smart Access System

Hardware Setup

- **Arduino Uno** – main controller.
- **RFID Reader (MFRC522)** – connected via SPI (SS → D10, RST → D8).
- **IMU Sensor (MPU6050)** – connected via I²C (SDA → A4, SCL → A5).
- **Servo Motor** – connected to D9 for physical access control.
- **LEDs** – green LED (D4) for “Access Granted,” red LED (D3) for “Access Denied.”
- All components share common 5V and GND connections.

Software

Arduino:

1. Initialize all sensors and modules (RFID, IMU, Servo, LEDs).
2. Continuously read IMU data (ax, ay, az) and check for RFID card presence.
3. When an RFID tag is detected, send the UID to Python via serial communication.
4. Wait for a response ('A' or 'D') from Python to control the servo and LEDs.

- 'A': servo rotates to open, green LED ON.
- 'D': servo remains closed, red LED ON.

Python side:

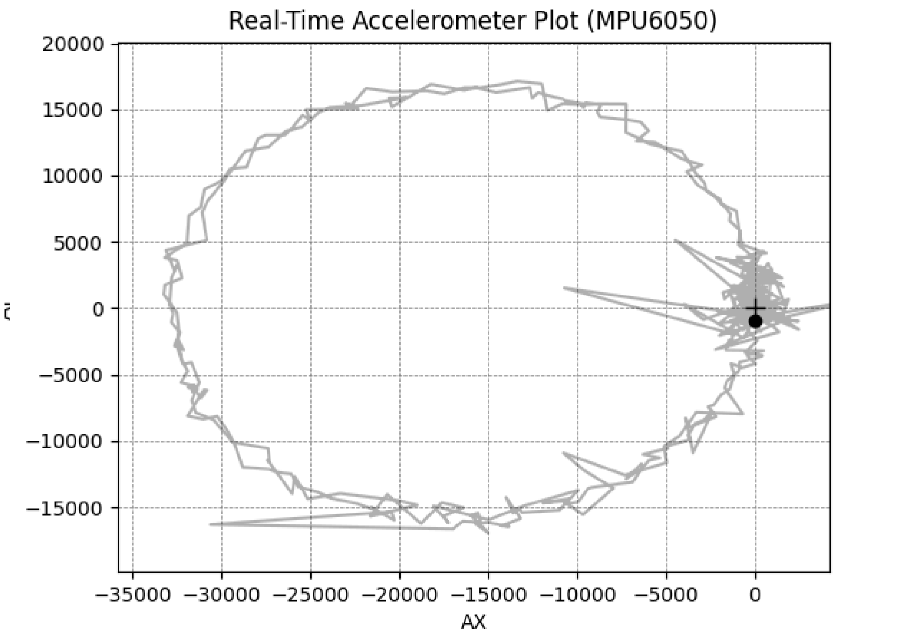
1. Establish serial communication with Arduino.
2. Read the RFID UID and compare it with the authorized UID list.
3. If authorized, collect IMU data for a short time window (e.g., 3 seconds).
4. Analyze motion data to detect a circular hand motion.
5. Send 'A' (Access Granted) if the motion is valid; otherwise, send 'D' (Access Denied).
6. Save all trial data (UID, IMU readings, decision) into CSV files.

Experimental Procedure

1. Connect all components and upload the Arduino code.
2. Run the Python program and open the serial port.
3. Place an RFID tag near the reader to detect and display its UID.
4. For an authorized tag, perform the required circular motion while the IMU collects data.
5. Observe the servo and LED response:
    - Green LED + servo open → correct tag and motion.
    - Red LED + servo closed → wrong tag or incorrect motion.
6. Repeat the test with multiple tags and motions to collect sufficient data for analysis.
7. Save all logs for performance evaluation (e.g., success rate, latency).

Data Collection

In this experiment, the MPU6050 sensor recorded acceleration values along the X and Y axes while being connected to an Arduino Mega. The Arduino sent the raw data through the serial port at a baud rate of 9600, and Python continuously received and plotted the data in real time.



When the data were plotted live using Python, the scatter points slowly formed a circular shape, illustrating how the acceleration varied as the sensor was rotated in a circular motion.

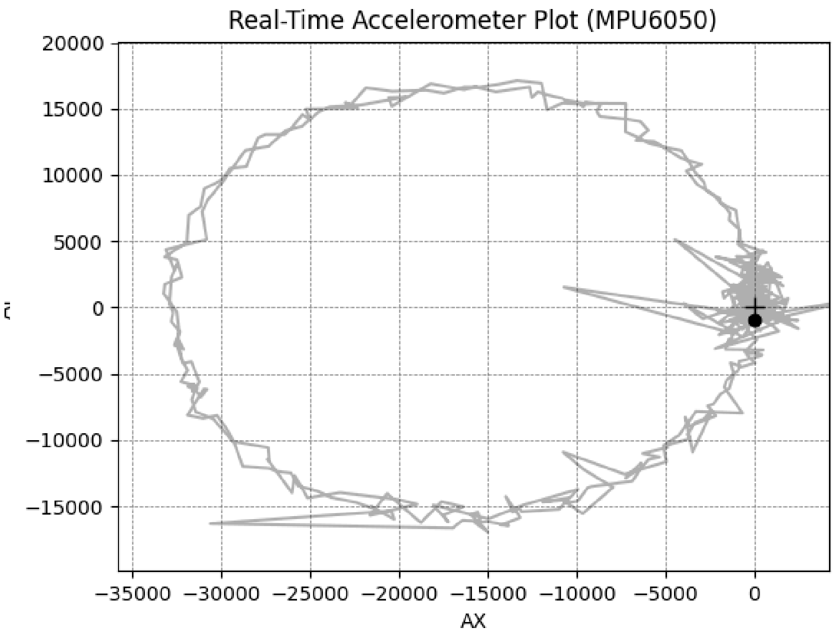
Data Analysis

- The MPU6050 sensor measures acceleration along the X, Y, and Z axes.
- Arduino sends the raw acceleration data to the computer via serial communication.
- In Python, the raw values are converted to acceleration in m/s² using the formula: a=(raw value/16384.0)×9.81a = (\text{raw value} / 16384.0) \times 9.81a=(raw value/16384.0)×9.81.
- A real-time scatter plot of X versus Z acceleration is generated using Matplotlib.
- When the sensor moves, the data points spread out; when it is still, the points cluster together.
- The system operates smoothly and reliably, updating the data continuously.
- Overall, this setup successfully visualizes the MPU6050's motion data in real time.

Results

For **Task 1**, the MPU6050 accelerometer successfully transmitted real-time motion data to the Python interface through serial communication. The resulting plot, as shown in figure below , illustrated a clear circular trajectory of the hand motion, confirming that the sensor accurately detected and visualized movement in two dimensions. The plotted path showed consistent circularity with slight deviations caused by natural hand jitter or transient noise at the beginning and end of motion. The live plot updated smoothly and effectively demonstrated the system's ability to track motion patterns in real time.

For **Task 2**, the integration of the RFID reader, MPU6050 sensor, servo motor, and LED indicators functioned as intended. When an authorized RFID tag was scanned, the system prompted the user to perform a circular motion. Upon successful gesture detection, the servo motor rotated 180 degrees to unlock the mechanism, while the green LED illuminated to signal access granted. In contrast, when an unauthorized card was scanned or when the circular motion failed to be detected, the system denied access and activated the red LED. The serial monitor displayed clear feedback messages confirming successful RFID authentication and motion verification. The results demonstrated that both the RFID and motion-based components worked reliably and synchronized effectively through Python-Arduino serial communication.


Real-Time Accelerometer Plot (MPU6050)
20000
15000
10000
5000
0
−5000
−10000
−15000
−35000
−30000
−25000
−20000
−15000
−10000
−5000
0
AX

Discussion

The **Task 1** results confirm that the MPU6050 sensor can effectively capture and represent real-time hand movement data. The observed circular pattern verified that the sensor's X and Y accelerometer readings were accurate enough to trace defined motion paths. Minor fluctuations in the plot were due to natural sensor noise and unsteady hand motion. These could be reduced by implementing filtering techniques such as a moving average or complementary filter. The responsiveness of the plotted data also reflected efficient serial communication and proper data parsing in Python. Overall, the experiment achieved its goal of real-time motion tracking and visualization with minimal error.

For **Task 2**, the integration of RFID authentication and motion verification demonstrated the feasibility of a dual-factor access control system. The expected system behavior—granting access only when both RFID and motion were verified—was achieved without delay or communication errors. The combination of hardware authentication (RFID) and behavior verification (motion gesture) provided enhanced security against unauthorized access. The experiment also showed how the Arduino and Python programs could operate collaboratively, where Arduino handled hardware control (servo and LEDs), while Python performed logical verification and pattern analysis. Any minor delay in serial response was negligible and did not affect system performance. The successful operation of this integrated system proves that real-time sensor data can be effectively used to enhance the security and intelligence of automated access mechanisms.

Recommendations

For **Task 1**, several improvements can be made to enhance the accuracy and smoothness of the real-time motion tracking system. One important recommendation is to implement a digital filtering method, such as a moving-average or Kalman filter, to reduce sensor noise and vibration effects that cause irregular lines in the plotted motion path. Using gyroscope data in combination with accelerometer readings could also improve stability and accuracy by compensating for drift and sudden acceleration spikes. Additionally, the visualization can be enhanced by including automated motion-pattern recognition, allowing the program to detect circular gestures or other predefined shapes without manual observation. For better analysis, data logging features could be added to store readings for post-processing and performance evaluation. Calibrating the MPU6050 before every session is also advised to ensure consistent sensor sensitivity and minimize offset errors.

For **Task 2**, future improvements should focus on expanding the system's functionality and robustness. The RFID + motion integration could be enhanced by storing authorized user IDs directly in the Arduino's EEPROM or in a cloud database to enable scalability and remote management. The motion-verification algorithm could be refined by incorporating threshold analysis or machine-learning-based gesture recognition for higher accuracy and adaptability to individual users' movement patterns. The use of a more powerful microcontroller, such as an ESP32, would allow wireless communication via Wi-Fi or Bluetooth, enabling mobile notifications or remote access control. Power management could also be optimized by supplying an external regulated 5 V source for the servo motor to prevent voltage drops during operation. Lastly, the physical system design could be improved by creating a compact housing and using proper cable management to make the setup more practical and durable for real-world implementation.

References

1. Last Minute Engineers. (2018, July 30). *What is RFID? How It Works? Interface RC522 RFID Module with Arduino*. Last Minute Engineers; Last Minute Engineers. https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/
2. Santos, R. (2016, March 23). *MFRC522 RFID Reader with Arduino Tutorial | Random Nerd Tutorials*. Random Nerd Tutorials. https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduin
3. Staff, L. E. (2020, December 29). *Interface MPU6050 Accelerometer and Gyroscope Sensor with Arduino*. Last Minute Engineers. https://lastminuteengineers.com/mpu6050-accel-gyro-arduino-tutorial
4. Singh, R., Kumar, P., & Kumar, S. (2020). *RFID based security and access control system using Arduino and MFRC522 module*. *International Research Journal of Engineering and Technology (IRJET)*, 7(6), 258–261. https://www.irjet.net/archives/V7/i6/IRJET-V7I6180.pdf

Student's Declaration

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.
We hereby certify that this report has not been done by only one individual and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate.
We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.
We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Signature : *Aiman*

Name : Muhammad Aiman Alfaruq Bin Noredan

Matric number : 2316963

Contributions : Introduction, Material and Equipment, and Experimental Setup

Signature : *Hariz*

Name : Muhammad Hariz bin Helmi Shamsul

Matric number : 2316441

Contributions : Results, Discussions and Conclusion

Signature : *syazmieakmal*

Name : Wan Syazmie Akmal bin Wan Saad

Matric number : 2316957

Contributions : Methodology, Data Collection and Data Analysis